

Методические указания к лабораторной работе №1

Понятие класса, структура класса (поля, свойства, методы), статические методы, объекты в качестве параметров методов, возврат объектов из методов.

Разработка программы в рамках объектно-ориентированной парадигмы (ООП) начинается с анализа предметной области задачи (домена), поиска сущностей предметной области (объектов), их атрибутов (свойств), характеризующих состояние объектов, а также присущего объектам поведения (действий, реакций, изменений состояний). Объекты или сущности группируются в классы или абстракции (типы данных).

Успех программы в ООП прежде всего заключается в грамотно спроектированной системе классов и отношений между ними. Алгоритм программы вторичен по отношению к архитектуре и рождается во взаимодействии объектов.

Чтобы корректно из домена выделить абстракции, т.е. типы данных, переменные которых (модели) есть объекты, необходимо:

- проанализировать задачу;
- найти в ней имена существительные – это и есть объекты.

Если объекты имеют единое поведение и набор атрибутов, то они принадлежат одной абстракции или типу данных (классу).

Говоря про поведение, подразумеваются алгоритмы (методы), которые обрабатывают данные в виде полей текущего класса (свойств), либо вложенных в него сущностей.

Рассмотрим примеры.

Квадрат, круг, прямоугольник, если это конкретные фигуры с заданными координатами, то это объекты. Они отличаются по свойствам, у квадрата все стороны равны, достаточно одного поля, у круга есть радиус, у прямоугольника две разные стороны. Значит, квадрат, круг и прямоугольник – это классы или типы данных для конкретных фигур. Можно, конечно, найти общие для всех классов поля и методы и выделить в базовую абстракцию, но это оставим для следующих ЛР.

Любые действия, описанные в задании, реализуются в методах классов, но так, чтобы относиться именно к конкретному классу и задействовать только его свойства.

Перемещение квадрата на плоскости – метод, меняющий состояние объекта, его свойства (координата центра), такие методы принадлежат объектам, т.к. в своей реализации используют их текущее состояние.

Далее представлен класс **Квадрат**. Он содержит два конструктора, поля координат верхней левой вершины, длину стороны, методы перемещения и изменения размера. Добавлен метод проверки пересечения фигур, причем, проверить пересечение квадрата можно с любой фигурой, для этого объект передается в метод как объект, а в методе проверяется его настоящий тип.

```

public class Square
{
    private double _x0, _y0, _len;

    public Square(double len) { _len = len; }
    public Square(double x, double y, double len) : this(len)
    {
        _x0 = x;
        _y0 = y;
    }

    public void Move(double x, double y)
    {
        _x0 += x;
        _y0 += y;
    }
    public void Resize(double len)
    {
        _len = len;
    }

    public override string ToString()
    {
        return $"Square ({_x0}, {_y0}, {_x0 + _len}, {_y0 + _len})";
    }

    public bool Intersects(object shape)
    {
        if (shape is Square)
        {
            var s = (Square) shape;

            return _x0 <= s._x0 && _y0 <= s._y0
                && _x0 + _len >= s._x0 && _y0 + _len >= s._y0;
        }

        if (shape is Rectangle)
        {
            var r = (Rectangle) shape;
            throw new NotImplementedException();
        }

        throw new NotSupportedException();
    }
}

```

Упрощенный класс **Прямоугольник**:

```

public class Rectangle
{
    private double x0, y0, len, width;

    public Rectangle(double x, double y, double l, double w)
    {
        /* ... */
    }
}

```

Пример работы:

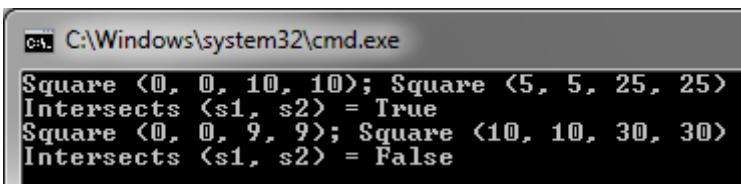
```
internal class Program
{
    private static void Main(string[] args)
    {
        var s1 = new Square(0, 0, 10);
        var s2 = new Square(5, 5, 20);

        bool res1 = s1.Intersects(s2);
        Console.WriteLine($"{s1}; {s2}");
        Console.WriteLine($"Intersects (s1, s2) = {res1}");

        s1.Resize(9);
        s2.Move(5, 5);

        bool res2 = s1.Intersects(s2);
        Console.WriteLine($"{s1}; {s2}");
        Console.WriteLine($"Intersects (s1, s2) = {res2}");

        Console.ReadKey();
    }
}
```



```
C:\Windows\system32\cmd.exe
Square <0, 0, 10, 10>; Square <5, 5, 25, 25>
Intersects <s1, s2> = True
Square <0, 0, 9, 9>; Square <10, 10, 30, 30>
Intersects <s1, s2> = False
```

Методы бывают статическими, они аналогичны функциям языка Си. Удобно в подобных методах реализовывать алгоритмы, которые все, что им нужно, берут из входных или формальных параметров. Например, определение факта (bool) пересечения фигур, либо создание новой фигуры (объекта), как результата пересечения двух фигур. Хотя, возможно реализовать такой алгоритм в обычном методе, для этого в качестве первого аргумента взять сам объект (ссылка this), а второй объект передать в метод.

Класс может выступать в роли контейнера вложенных объектов и его методы будут подобны операциям с массивами или списками.

Есть два подхода:

- *Вложенные объекты имеют самостоятельный тип данных.*
Объекты поступают в контейнер и возвращаются из контейнера, как самостоятельные сущности.
- *Тип данных объектов инкапсулирован в классе.*
Тип данных скрыт внутри класса, а ввод-вывод осуществляется в виде передачи отдельных свойств вложенных объектов.

В любом случае, методы контейнера контролируют весь ввод-вывод и поддерживают внутренние структуры класса в непротиворечивом состоянии.

Рассмотрим **контейнер строк** (Strings), содержащий список строк и умеющий объединять строки без повторений с возвратом нового объекта-контейнера.

```
public class Strings : IEnumerable<string>
{
    private readonly List<string> _list = new List<string>();

    public Strings(IEnumerable<string> values = null)
    {
        if (values != null) _list.AddRange(values);
    }

    public int Count { get { return _list.Count; } }

    public void Add(string value) { _list.Add(value); }
    public void Clear() { _list.Clear(); }

    public IEnumerator<string> GetEnumerator()
    {
        return _list.GetEnumerator();
    }

    IEnumerator IEnumerable.GetEnumerator()
    {
        return GetEnumerator();
    }

    public static Strings Union(Strings s1, Strings s2)
    {
        if (s1 == null) throw new ArgumentNullException(nameof(s1));
        if (s2 == null) throw new ArgumentNullException(nameof(s2));

        var set = new HashSet<string>(s1);
        set.UnionWith(s2);

        return new Strings(set);
    }
}
```

Класс содержит внутренний список строк `_list` для хранения. Кроме того, сам класс `Strings` содержит статический метод `Union()`, объединяющий два контейнера строк в один новый контейнер. Для исключения повторений строк можно воспользоваться готовой коллекцией `HashSet<>` и реализованным в ней методом `UnionWith()`. Полезно, но не обязательно, строковый контейнер наследовать от обобщенного интерфейса перечислителя `IEnumerable<string>` с реализацией метода перечисления строк `GetEnumerator()`, в котором просто делегировать этот перечислитель списку. Поддержка перечислителя позволяет задействовать цикл `foreach` по внутренним строкам контейнера и выполнить сопряжение с `HashSet<>` в его конструкторе и параметре метода `UnionWith()`.

Конструктор класса `Strings` либо не принимает никаких аргументов (является конструктором по умолчанию), либо принимает любую объектную коллекцию строк (массив, список) для первичного заполнения внутреннего списка. Подобная реализация конструктора не обязательна, но позволяет использовать встроенный в язык синтаксис инициализации объекта коллекцией в фигурных скобках:

```

var s1 = new Strings {"abc", "def", "abc", "ghi"};
var s2 = new Strings {"def", "ghi", "jkl"};

var s3 = Strings.Union(s1, s2);

foreach (string s in s3)
    Console.WriteLine(s); // abc def ghi jkl

```

Рассмотрим класс **Записная книжка**. Каждая запись пусть будет объектом класса Note.

```

public class Note
{
    public string FullName { get; set; }
    public DateTime BirthDate { get; set; }
    public string Phone { get; set; }

    public override string ToString()
    {
        return $"{FullName}, "
            + $"{BirthDate:d}, "
            + $"{Phone ?? "null"}";
    }
}

```

Метод ToString() реализован для удобства вывода объекта на консоль. Он автоматчиками вызывается для объекта, передаваемого в консольные методы вывода информации.

Записная книжка хранит список записей (List). Реализован метод поиска по номеру телефона с возвратом null, если запись не найдена. Сортировка по ФИО осуществляется стандартным методом списка Sort() с передачей в него компаратора сравнения пар строк (FullNameComparer).

```

public class Notebook
{
    private readonly List<Note> _notes = new List<Note>();

    private class FullNameComparer : IComparer<Note>
    {
        public int Compare(Note x, Note y)
        {
            return string.Compare(
                x.FullName, y.FullName,
                StringComparison.CurrentCulture);
        }
    }

    public void Add(Note note)
    {
        if (note == null)
            throw new ArgumentNullException(nameof(note));

        if (string.IsNullOrEmpty(note.FullName))
            throw new ArgumentNullException(nameof(note.FullName));

        _notes.Add(note);
    }
}

```

```

public void SortByFullName()
{
    if (_notes.Count == 0) return;

    _notes.Sort(new FullNameComparer());
}

public Note FindByPhone(string phone)
{
    foreach (var note in _notes)
        if (note.Phone == phone) return note;

    return null;
}

public Note[] GetAll()
{
    return _notes.ToArray();
}
}

```

Заполним записную книжку тестовыми записями и выполним сортировку и поиск.

```

var notebook = new Notebook();

notebook.Add(new Note
{
    FullName = "Волков Артём Денисович",
    BirthDate = new DateTime(2002, 4, 7),
    Phone = "8(913)356-33-68"
});
notebook.Add(new Note
{
    FullName = "Александров Мирон Григорьевич",
    BirthDate = new DateTime(2001, 5, 5),
    Phone = "8(912)354-44-67"
});
notebook.Add(new Note
{
    FullName = "Ларин Ян Алексеевич",
    BirthDate = new DateTime(2005, 1, 13),
    Phone = "8(916)908-15-26"
});
notebook.Add(new Note
{
    FullName = "Крылова Алиса Владимировна",
    BirthDate = new DateTime(2004, 2, 11),
    Phone = "8(915)543-78-12"
});
notebook.Add(new Note
{
    FullName = "Ермакова Ника Кирилловна",
    BirthDate = new DateTime(2003, 3, 9),
    Phone = "8(914)753-72-16"
});

```

```

Console.WriteLine(notebook.FindByPhone("8(912)354-44-67"));
// (Александров Мирон Григорьевич, 05.05.2001, 8(912)354-44-67)

notebook.SortByFullName();

foreach (var note in notebook.GetAll())
    Console.WriteLine(note);

// (Александров Мирон Григорьевич, 05.05.2001, 8(912)354-44-67)
// (Волков Артём Денисович, 07.04.2002, 8(913)356-33-68)
// (Ермакова Ника Кирилловна, 09.03.2003, 8(914)753-72-16)
// (Крылова Алиса Владимировна, 11.02.2004, 8(915)543-78-12)
// (Ларин Ян Алексеевич, 13.01.2005, 8(916)908-15-26)

```

В результате выполнения ЛР студент должен уметь: создавать классы, правильно оценивать их количество, внутреннюю структуру (поля и методы), создавать статические методы, если они нужны, уметь передавать объекты в методы и создавать новые объекты, как результат работы метода.

Важно при проектировании класса сделать его независимым от той инфраструктуры, в которой он «живет», прежде всего, убрать из класса консольный ввод-вывод. Подобная практика позволит в дальнейшем сделать код классов переносимым в другое окружение (оконное, сетевое).

Во время проверки ЛР преподаватель делает основной акцент на исходном коде классов, тогда как сама работа с объектами классов, меню программы, ввод-вывод, проверка ввода – вторична и нужна лишь для демонстрации взаимодействия объектов, вызова их методов.